

JARVIS MARK-I — Day 2 Code & Workflow Explanation

This document explains the complete Day 2 workflow, code structure, modules, functions, and system architecture of the JARVIS MARK-I AI assistant.

Complete Workflow

YOU SPEAK → Microphone Records Audio → Google Speech Recognition Converts Voice to Text → Python Checks Commands → Jarvis Decides Action → edge-tts Converts Text to AI Voice → pygame Plays Audio → Jarvis Responds

Modules Used

speech_recognition — Converts speech into text **asyncio** — Handles asynchronous programming **edge_tts** — Generates AI voice **pygame** — Plays audio files **os** — Handles file operations **webbrowser** — Opens websites automatically **datetime** — Retrieves current system time

Day 2 Main Code

```
import speech_recognition as sr
import asyncio
import edge_tts
import pygame
import os
import webbrowser
import datetime

recognizer = sr.Recognizer()

async def speak(text):

    print(f"\n[JARVIS]: {text}")

    communicate = edge_tts.Communicate(
        text=text,
        voice="en-US-GuyNeural"
    )

    await communicate.save("voice.mp3")

    pygame.mixer.init()
    pygame.mixer.music.load("voice.mp3")
    pygame.mixer.music.play()

    while pygame.mixer.music.get_busy():
        continue

    pygame.mixer.quit()

    os.remove("voice.mp3")

def listen():

    with sr.Microphone() as source:

        print("\n[STATUS]: Listening...")
        recognizer.adjust_for_ambient_noise(source)
```

```

        audio = recognizer.listen(source)

        print("[STATUS]: Recognizing...")

        text = recognizer.recognize_google(audio)

        print(f"\n[YOU]: {text}")

        return text.lower()

print("=" * 50)
print("      JARVIS MARK-II")
print("=" * 50)

asyncio.run(
    speak("System online. How can I help you?")
)

command = listen()

if "youtube" in command:

    asyncio.run(
        speak("Opening YouTube")
    )

    webbrowser.open("https://youtube.com")

elif "google" in command:

    asyncio.run(
        speak("Opening Google")
    )

    webbrowser.open("https://google.com")

elif "time" in command:

    time = datetime.datetime.now().strftime("%I:%M %p")

    asyncio.run(
        speak(f"The current time is {time}")
    )

else:

    asyncio.run(
        speak("Command not recognized")
    )

```

Important Functions

Recognizer() — Creates speech recognition engine **Microphone()** — Accesses microphone input **listen()** — Records audio from microphone **recognize_google()** — Converts speech into text **Communicate()** — Generates AI speech **save()** — Saves generated speech as MP3 **pygame.mixer.init()** — Starts audio engine **music.load()** — Loads MP3 file **music.play()** — Plays generated audio **webbrowser.open()** — Opens websites automatically **datetime.now()** — Gets current time **strftime()** — Formats time nicely

System Architecture

The assistant currently contains: Input System — Microphone Processing System — Speech Recognition Decision System — Command Logic Output System — AI Voice Action System — Website Automation

Concepts Learned

Python Modules Functions Variables Conditional Statements Asynchronous Programming Speech Recognition AI Voice Generation Audio Processing Automation Error Handling